Master Build and Commercial Plan

From problem statement to paying customers · Phased, gated, sequential · October 2026

What this is. The master plan for building Eagle's Eye together, from an empty folder to a paying commercial product. Work runs in **sequential phases**. Each phase has deliverables, an exit gate and a decision at the end, so we never start phase N+1 on faith. **Division of labour:** Claude writes and tests the code, docs and site. The owner runs customer conversations, sales, accounts and budget, and makes the decisions flagged in section 14.

1. Problem statement

1.1 The problem

Businesses want visitors to get accurate answers from their own website at any hour. The people who build those websites, such as freelancers, agencies and in-house developers, have no **cheap, controllable, reliable way** to add that ability. Today they must either assemble six or seven separate systems (crawler, text extraction, search index, model hosting, conversation memory, business rules, widget and dashboard) or rent a hosted chatbot builder that charges per conversation, locks in their data and limits customisation.

1.2 Who has it

Segment	Pain	Priority
Freelance developers and small agencies	Rebuild the same chatbot stack for each client; cannot justify hosted per-seat fees across many sites	**Primary (first)**
SaaS and documentation owners	Users cannot find answers in large docs; support tickets repeat	Secondary
Small business owners (via developers)	No staff to answer repetitive questions; lost leads outside hours	Reached through the primary segment
Privacy-sensitive organisations	Cannot send visitor data to third-party AI vendors	Later (enterprise)

1.3 Why current options fall short

- **Hosted builders** (Intercom Fin, Botpress and similar): fast to start, but recurring usage fees, vendor dependency, limited control over deployment and behaviour.
- **DIY with a general LLM API:** flexible, but the developer owns crawling, retrieval quality, hallucination control, evaluation and maintenance, and it is slow to ship.
- **Self-hostable frameworks** (Dify and similar): broad and powerful, but general-purpose, so website-specific accuracy and quick setup are left to the user.

- **Plain keyword or rule chatbots:** predictable, but cannot handle natural language or follow-up questions.

1.4 Consequences of the problem

- Developers lose days or weeks per client integration.
- Visitors get wrong, generic or no answers, so trust and conversions fall.
- Businesses either overpay for hosted AI or ship nothing.

1.5 Our problem statement

> Developers who build websites for clients have no affordable, portable way to give those sites an assistant that answers accurately from the site's own content. They must either build a multi-system stack themselves or accept the cost, lock-in and limited control of hosted chatbot platforms.
>
> **Eagle's Eye** turns a website into a source-grounded, configurable conversational assistant that a developer can embed in minutes and run hosted or self-hosted, with answers that cite their sources and admit when they do not know.

1.6 Scope

In scope (v1)	Explicitly out of scope (v1)
Crawl and index public website content; retrieval; grounded answers with citations; abstention; conversation memory; embeddable widget; TypeScript SDK; REST API; hosted service with usage limits; self-host via Docker; marketing and docs website; developer dashboard; basic analytics	Training a foundation model; custom vector database; voice or phone; visual workflow builder; marketplace; authenticated or private-content crawling; live-agent handoff inbox; native mobile SDKs; SOC 2 audit (planned, not v1)

1.7 Key assumptions to test

#	Assumption	How it is tested	Phase
A1	Developers and agencies feel this pain strongly enough to switch tools	15–20 interviews	0
A2	Retrieval plus a small open-weight model can answer website questions accurately enough to ship	Evaluation set, target scores in section 8	1
A3	A developer can integrate in under 30 minutes	Timed external integrations	2, 5
A4	Hosted serving cost allows at least 70% gross margin at the planned price	Measured cost per resolved conversation	4, 6
A5	Some developers will pay for hosting and convenience even though the core is free	Paid pilots and conversion	6
A6	A fine-tuned model beats the base model enough to justify the effort	A/B on the evaluation set	7

1.8 Success and kill criteria

Success (end of Phase 6): at least 5 independent integrations, at least 3 paying customers, 30-day project retention of 40% or more among activated projects, and gross margin on hosted plans of 70% or more. **Kill or pivot triggers:** fewer than 5 of 20 interviewees describe the pain as serious; evaluation accuracy cannot reach the section 8 targets after two improvement cycles; nobody agrees to a paid pilot after 15 genuine offers. If a trigger fires we stop and reassess, not push on.

2. Product definition

Component	What it is
Engine (server)	Crawler, extractor, index, retrieval, conversation runtime, rules engine, model adapter, REST API
SDK	Typed TypeScript client for the API, usable in Node and browsers
Widget	Small embeddable chat component (script tag or React package), themeable, accessible
Dashboard	Create projects, add sites, review crawled pages, test questions, view analytics, manage keys and billing
Website	Marketing site, pricing, documentation, tutorials, blog, live demo, status and legal pages
Hosted service	Managed engine plus model inference, indexing, quotas and billing
Self-host kit	Docker Compose and documented deployment profiles

Answer quality principles

- **Grounded:** every factual answer is built from retrieved passages and links to them.
- **Abstaining:** if evidence is weak, say so and offer a fallback (contact, search link). Never invent.
- **Deterministic where it matters:** prices, hours, eligibility and calculations come from structured rules or extracted fields, not free generation.
- **Safe:** website text is treated as untrusted data, never as instructions to the model.
- **Measurable:** every change is scored against a fixed evaluation set.

3. Architecture and technology choices

Choices favour few moving parts, replaceable interfaces and the owner's existing stack (TypeScript, React, Postgres, Vercel, Render, Supabase, Cloudflare). All are defaults we can revisit at a phase gate.

Layer	Choice	Reason
Repo	pnpm monorepo: packages/ core, crawler, sdk, widget; apps/ api, dashboard, site, docs	One place, shared types, simple CI
Language	TypeScript end to end	Single skill set; typed SDK is the product
API	Fastify (or NestJS if preferred), OpenAPI schema generated	Lightweight, fast, typed contracts

Layer	Choice	Reason
Database	PostgreSQL with built-in full-text search (BM25-style ranking)	No extra service in v1; pgvector added only if evaluation shows semantic search helps
Crawler	HTTP fetch with Playwright fallback for JavaScript pages; robots.txt respected; readability-style extraction	Covers static and SPA sites
Model interface	Provider-agnostic adapter: Ollama and vLLM (self-host), OpenAI-compatible endpoints (hosted)	Same config hosted or self-hosted
Base model	Benchmark 2–3 open-weight instruct models of 7–14B size on our evaluation set, then pick	Decide on data, not hype
Widget	Preact or vanilla web component, under 30 KB gzip, shadow DOM	Light, style-isolated, embeddable anywhere
Website	Astro for marketing and docs (fast, SEO-friendly); React/Vite for the dashboard	Right tool for each job
Hosting	Vercel (site, dashboard), Render (API, workers), Supabase (Postgres, auth), R2 (storage), GPU inference rented per use	Low-ops, consistent with earlier AIIT decision
Billing and email	Stripe, Resend	Standard
Quality gates	GitHub Actions: typecheck, tests, evaluation run, build, container scan	Regressions caught automatically

Request flow

Visitor message → widget → API (auth, rate limit) → conversation context → query rewriting for follow-ups → retrieval over the project's index → rules and structured-field lookup → model generates a grounded answer from the evidence → validator checks citations and policy → response with sources, or abstention → usage logged.

Core data model

Organisation, Project, Site, Page (with content hash and version), Chunk (text, heading path, URL, position), CrawlJob, KnowledgeVersion (for rollback), Conversation, Message, Feedback, Rule, ApiKey, UsageEvent, Plan and Subscription.

Public API surface (v1)

- POST /projects, POST /projects/:id/sites, POST /projects/:id/crawl, GET /crawl/:id
- GET /projects/:id/pages, POST /projects/:id/search (retrieval only, for debugging)
- POST /projects/:id/chat with streaming; POST /feedback
- GET /projects/:id/usage, API-key management, webhooks for crawl completion

4. Phased roadmap

Durations assume steady weekly progress by two collaborators and are **planning targets**, not promises. Phase 0 starts the week of 12 October 2026. Each phase ends at a gate: we review evidence together and decide to proceed, repeat or stop.

Phase	Name	Target window	Gate to pass
0	Foundations and validation	Weeks 1–3	Pain confirmed in interviews; repo and CI live
1	Core engine	Weeks 3–8	Evaluation targets met on test sites
2	SDK and widget	Weeks 8–10	Developer integrates in under 30 minutes
3	Website and documentation	Weeks 9–12 (overlaps 2)	Site live; docs complete; waitlist open
4	Hosted service and dashboard	Weeks 11–16	Multi-tenant hosted API with quotas and cost metering
5	Private alpha	Weeks 16–20	5+ external integrations without our help
6	Public beta and monetisation	Weeks 20–28	3+ paying customers; margin and retention targets
7	Specialised model	Weeks 26–36	Fine-tuned model beats baseline on cost or quality
8	Scale and enterprise	Week 36 onward	Driven by demand, not the calendar

Phase 0 — Foundations and validation (Weeks 1–3)

Goal: confirm the problem is real and set up a sound engineering base.

- **Owner:** interview 15–20 developers and agencies (script in section 9); list 30 target prospects; secure domain, GitHub organisation, Stripe and cloud accounts; decide licence direction (section 14).
- **Claude:** scaffold monorepo, TypeScript config, lint, test runner, CI, container setup, ADR (architecture decision record) folder, evaluation harness skeleton; write the interview synthesis template; assemble 5 real test websites of different types.
- **Deliverables:** repo with green CI, interview findings document, chosen name and domain, licence decision, test-site list.
- **Gate:** at least 10 interviews done and the kill triggers in 1.8 not fired.

Phase 1 — Core engine (Weeks 3–8)

Goal: given a URL, produce trustworthy, cited answers. This is the technical heart, so it gets the most care.

- **Crawler and extractor:** sitemap and link discovery, robots.txt, rate limiting, canonical URL handling, de-duplication, boilerplate removal, headings preserved, JavaScript rendering fallback, incremental recrawl by content hash.

- **Indexer:** structure-aware chunking by heading, metadata (URL, title, section, date), Postgres full-text index, knowledge versioning.
- **Retrieval:** ranked search, query rewriting for follow-ups, top-k selection with diversity; optional embeddings tested against the evaluation set before adoption.
- **Conversation runtime:** session memory, grounded prompt templates, citation extraction, validator, abstention threshold, fallback behaviour.
- **Rules layer:** structured extraction of prices, hours, contact details; configurable rules and canned responses that override generation.
- **Model adapter:** Ollama and an OpenAI-compatible endpoint behind one interface; benchmark candidate models.
- **Evaluation harness:** 150–300 question set across 5 sites (see section 8), automatic scoring, report per run, CI gate.
- **Security:** prompt-injection tests using hostile page content; SSRF protection in the crawler (block private IP ranges); size and time limits.
- **Gate:** targets in section 8 met on test sites; crawl of a 500-page site completes reliably; no critical security test failing.

Phase 2 — SDK and widget (Weeks 8–10)

- Typed TypeScript SDK generated from the OpenAPI schema; Node and browser builds; streaming support.
- Widget: script-tag and React package, theming tokens, keyboard and screen-reader accessibility, mobile layout, citation links, feedback buttons, safe markdown rendering, domain allow-list.
- Example apps (plain HTML, React, Next.js) and a local-run quick start.
- **Gate:** three people outside the team time a first integration, median under 30 minutes.

Phase 3 — Website and documentation (Weeks 9–12)

The website is a product, not decoration. It is how developers discover, trust and adopt Eagle's Eye, so it is built as its own phase, starting once the engine is demonstrable.

Page or area	Purpose and content
Home	Problem statement in one screen; live demo (enter a URL, ask a question, see cited answers); three proof points; install snippet; clear call to action
Product	How it works: crawl, index, retrieve, answer, embed; hosted versus self-hosted; security and privacy summary
Pricing	Community, Developer Pro, Business Hosted, Enterprise; usage allowances; FAQ on overage and self-hosting
Docs	Quick start, concepts, API reference (generated), SDK reference, widget theming, self-host guide, troubleshooting, changelog
Tutorials and blog	Task-based guides tied to working repositories; engineering write-ups on evaluation results
Compare	Honest comparison pages once we have measured facts

Page or area	Purpose and content
Trust	Privacy policy, terms, data processing terms, security page, crawler user-agent and opt-out page, status page
Waitlist and contact	Email capture feeding the design-partner pipeline

- **Design:** one coherent brand (name, logo, colour, type), light and dark themes, responsive, accessible (WCAG 2.1 AA target), fast (Core Web Vitals in the green).
- **SEO and analytics:** semantic markup, sitemap, structured data, privacy-respecting analytics, conversion events (demo run, install, signup).
- **The demo is the best salesperson.** It must run on any public URL within a safe rate limit and show sources.
- **Gate:** site live on the production domain, docs verified by following them from scratch, demo works on ten unfamiliar sites, waitlist collecting.

Phase 4 — Hosted service and dashboard (Weeks 11–16)

- **Multi-tenancy:** organisations, projects, API keys, row-level isolation, per-project quotas and rate limits.
- **Dashboard:** sign-up and auth, add site, crawl progress, page browser, test console with sources, rules editor, widget configuration and install snippet, usage and cost view, feedback review.
- **Inference:** start with an OpenAI-compatible hosted open-weight provider or per-use rented GPU. Commit to dedicated hardware only when sustained usage justifies it.
- **Metering and billing:** usage events (requests, tokens, pages indexed, crawls); Stripe subscriptions with included allowances and overage rules.
- **Operations:** backups, migrations, structured logs, error tracking, uptime checks, alerting, runbooks, cost dashboard.
- **Gate:** two isolated test tenants, load test at planned concurrency, cost per resolved conversation measured, billing tested end to end in Stripe test mode.

Phase 5 — Private alpha (Weeks 16–20)

- Invite 5–10 design partners from interviews and the waitlist; give each a named contact and a weekly check-in.
- Instrument activation funnel: signup, first crawl, first answer, embed, 7-day and 30-day activity.
- Triage every failure: crawl problems, wrong answers, integration friction. Feed failures into the evaluation set.
- Security review and a third-party dependency and container audit; privacy policy reviewed by a professional.
- **Gate:** at least 5 integrations completed without our hands-on help; top ten failure causes fixed or documented.

Phase 6 — Public beta and monetisation (Weeks 20–28)

- Open signup; launch on GitHub, npm, Hacker News, relevant developer communities and tutorials.
- Introduce paid plans; run price tests (two price points per plan) and paid pilots with agencies.

- Publish the evaluation report openly; honesty here builds trust.
- Customer support routine: documented response times, public issue tracker, changelog cadence.
- **Gate:** 3 or more paying customers, margin 70% or more on hosted plans, retention targets met, support load per customer sustainable.

Phase 7 — Specialised model (Weeks 26–36)

- Use real, consented, de-identified conversation and failure data plus reviewed synthetic data to build the training set.
- Parameter-efficient fine-tuning (LoRA-style) on the chosen base for structured extraction, tool selection, grounded concise answers and abstention.
- A/B against the base model on the held-out evaluation set and in shadow mode on live traffic.
- Quantise and benchmark serving cost; add routing between small and larger models.
- **Gate:** adopt only if quality is at least equal and cost lower, or quality clearly higher at acceptable cost. Otherwise keep the base model; the plan does not depend on a custom model.

Phase 8 — Scale and enterprise (Week 36 onward)

- Docker Compose and Kubernetes self-host profiles with support tiers.
- SSO, role-based access, audit exports, data residency options.
- More connectors (common CMS platforms, documentation generators), authenticated-content crawling, human handoff.
- SOC 2 readiness if enterprise demand justifies it.
- Agency programme: multi-client management, reseller pricing, white labelling.

5. Work breakdown for Phase 0 and Phase 1

The first concrete weeks, so we can start immediately after approval.

Week	Claude builds	Owner does
1	Monorepo scaffold, CI, lint, tests, container baseline, ADR-001 (stack), five test sites chosen, evaluation harness skeleton	Book interviews; claim domain and GitHub org; decide licence direction
2	Crawler v0 (fetch, robots, sitemap, rate limit, SSRF guard); extraction; page store	Run interviews; send interview synthesis
3	Chunking, Postgres full-text index, search endpoint; first evaluation questions written	Finish interviews; Phase 0 gate review
4	Model adapter (Ollama and OpenAI-compatible); first grounded answers with citations	Provide sample client sites if available
5	Query rewriting, conversation memory, abstention; evaluation run #1 and baseline report	Review baseline report; adjust targets
6	Rules layer and structured fields (prices, hours, contacts); JS-rendering fallback	Line up design-partner candidates
7	Model benchmark across candidates; semantic-search experiment; injection and SSRF tests	Choose base-model direction with Claude

Week	Claude builds	Owner does
8	Hardening, 500-page crawl test, docs for engine internals, Phase 1 report	Phase 1 gate review

6. Quality and evaluation plan

Evaluation set

- 150–300 questions across at least 5 websites of different types (service business, e-commerce, documentation, institution, SaaS).
- Mix: direct facts, paraphrases, multi-turn follow-ups, ambiguous questions, questions with no answer on the site, adversarial and injection attempts, rule-based facts (prices, hours).
- Each item has the expected answer, the source page, and whether abstention is correct. Reviewed by a human, not only generated.

Phase 1 target scores (to be confirmed at Phase 0 gate)

Metric	Target
Answer correctness on answerable questions	85% or higher
Citation validity (cited page actually supports the answer)	95% or higher
Correct abstention on unanswerable questions	90% or higher
Hallucinated facts on rule-based fields (prices, hours)	0 in the set
Prompt-injection test pass rate	100% on the defined suite
Median response latency (hosted, first token)	under 2 seconds
Crawl success on supported public sites	95% or higher of reachable pages

These are starting targets. If they prove unrealistic we say so openly and adjust with evidence, not by quietly lowering the bar.

7. Security, privacy and legal

- **Crawling ethics:** honour robots.txt, identify with a documented user-agent, rate-limit politely, provide an opt-out. Only crawl sites the customer owns or is authorised to index; require acceptance of that in terms.
- **Prompt injection:** retrieved text is wrapped as data; instructions in pages are ignored; tool calls restricted to an allow-list; outputs validated.
- **Infrastructure:** SSRF protection, secrets in a vault, least-privilege keys, TLS everywhere, dependency and container scanning, rate limiting, abuse monitoring.
- **Privacy:** collect the minimum conversation data; configurable retention and deletion; no use of customer data for training without explicit opt-in; clear subprocessor list; data processing agreement for business customers. GDPR and similar rules apply to visitor data, so get professional advice before launch.

- **Licensing:** choose the open-core licence deliberately (section 14); verify every dependency and model licence permits commercial and hosted use before shipping.
- **Legal documents needed before beta:** terms of service, privacy policy, acceptable use policy, data processing addendum, open-source licence and contributor terms. Have a qualified lawyer review them; this plan is not legal advice.

8. Commercial plan

8.1 Offer and pricing

Prices are **hypotheses to test**, not validated figures.

Plan	Price (test)	Includes
Community	Free	Self-hosted core, local development, basic indexing, community support
Developer Pro	$19–$39/mo	Multiple projects, hosted inference allowance, analytics, faster support
Business Hosted	$49–$149/mo	Higher allowances, automatic reindexing, branding, monitoring, production support
Private / Enterprise	Custom	Private deployment, access controls, support commitments, integrations

- Meter requests, tokens, indexed pages, storage and crawls; include allowances; clear overage; no unlimited hosted inference at launch.
- Add agency pricing (multiple client projects) after repeat usage is seen.
- Services: paid setup, integration, private deployment.

8.2 Unit economics

Cost per resolved conversation = (inference + retrieval + infrastructure + support) ÷ resolved conversations. Phase 4 measures it; Phase 6 manages it. **Gate:** hosted plans must reach at least 70% gross margin at the plan allowances, or we change allowances, price or model before scaling marketing. Illustrative revenue arithmetic: 50 × $29 = $1,450 per month; 100 × $49 = $4,900; 250 × $79 = $19,750. These are not forecasts.

8.3 Go-to-market sequence

Stage	Action	Success signal
Pre-launch	Interviews; waitlist; build in public with progress posts; demo videos	200+ waitlist signups, 20+ warm design-partner candidates (targets)
Alpha	Design partners onboarded personally	5 independent integrations
Launch	GitHub and npm release; Show HN and developer communities; three tutorials; comparison page	Activation rate and first paid conversions
Growth	Agencies programme; integrations on request; case studies; partner referrals	Weekly active projects rising; repeat deployments
Expansion	Enterprise and private deployments; reseller channel	Contracts above the self-serve ceiling

8.4 Funnel metrics

Visits → demo runs → signups → first crawl → first embed → 7-day active → 30-day active → paid. Review weekly; the weakest step gets the next sprint.

9. Customer discovery script

- How do you add chat or Q&A to client sites today? Walk me through the last one.
- How long did it take and what broke or cost the most?
- What do you pay for tools, hosting and model usage per client?
- Would you need self-hosting? Why? What data restrictions do clients impose?
- What would make you switch? What would stop you?
- Show the demo. Where does it fail you? Which answer would you not trust?
- Which features would you pay for? Would you pay now for a pilot at $X?

Record each interview, tag pains and objections, and record the price each person says they would pay. Do not pitch. Listen.

10. Team, tools and cadence

- **Cadence:** weekly plan and review; end-of-phase gate review with written evidence; decisions logged as ADRs.
- **Tracking:** GitHub Issues and Projects; one milestone per phase; the evaluation report is the single source of truth for quality.
- **Definition of done:** code reviewed, tests passing, evaluation not regressed, docs updated, security checks passed, changelog entry.
- **Branching:** trunk-based with short branches and pull requests; CI must be green to merge.

11. Budget outline

Indicative categories only. We replace these with real quotes at Phase 0.

Item	When	Notes
Domain, email, design assets	Phase 0	Low, one-off plus annual
Hosting (Vercel, Render, Supabase, R2)	From Phase 3	Free or low tiers at first
Inference (hosted open-weight or rented GPU)	From Phase 4	Largest variable cost; capped by quotas
Stripe fees	Phase 6	Percentage of revenue
Legal review of terms, privacy, licence	Before Phase 5	Do not skip
Security audit or penetration test	Before public beta	Scoped to the hosted service
Dedicated GPU hardware	Only after sustained paid demand	Avoid early commitment

12. Risk register

Risk	Likelihood	Impact	Mitigation	Owner
Product looks like another chatbot builder	High	High	Lead with portability, website compilation and measurable accuracy; validate wedge in interviews	Both
Answer quality below targets	Medium	High	Evaluation-driven iteration; retrieval tuning; abstention; limit claims	Claude
Hosted inference cost erodes margin	Medium	High	Quotas, small models, caching, price tests, per-use GPUs	Both
Crawl failures on JavaScript or protected sites	High	Medium	Rendering fallback; clear unsupported list; diagnostics	Claude
Prompt injection or data leakage	Medium	High	Isolation, validation, test suite, audit	Claude
Licence or legal problem	Low	High	Early legal advice; dependency and model licence checks	Owner
Developers use core free and never pay	Medium	High	Charge for hosting and recurring value; test payment early	Owner
Large vendor copies the idea	Medium	Medium	Move fast; own evaluation data and developer community	Both
Scope creep delays launch	High	Medium	Strict P0/P1/P2; gates; non-goals list	Both
Single-point dependency on one person's time	Medium	Medium	Documentation, ADRs, automation, clear cadence	Both

13. Success metrics by phase

Phase	Key measures
0	Interviews completed; share describing the pain as serious; CI green
1	Section 6 scores; crawl success; injection suite
2	Median time to first integration; widget size; accessibility audit result
3	Demo completion rate; docs task success; waitlist signups
4	Cost per resolved conversation; p95 latency; uptime; billing correctness
5	Independent integrations; activation rate; failure causes closed
6	Paying customers; MRR; free-to-paid; 30-day retention; gross margin; support time
7	Fine-tune versus baseline quality and cost
8	Enterprise pipeline; deployment success rate

14. Decisions needed from the owner

#	Decision	My recommendation
D1	Product name and domain: confirm Eagle's Eye and check availability and trademark	Confirm, then secure domain and GitHub org this week
D2	Open-core licence: permissive for SDK and widget; engine licence that protects against hosted resale	Apache-2.0 for SDK and widget; engine under AGPL-3.0 or a source-available licence; get legal advice first
D3	API framework: Fastify or NestJS	Fastify for lightness; NestJS if you prefer your AIIT conventions
D4	First test-site types and any real client sites we may use	Five mixed sites, including one of yours
D5	Inference approach for alpha: hosted open-weight API or rented GPU	Hosted open-weight API first, cheapest to start and easy to swap
D6	Monthly budget ceiling for hosting and inference	Set a hard cap; quotas enforce it
D7	Time commitment per week for customer outreach	At least 5 hours per week from Phase 0

15. How we start

- **Step 1:** approve this plan and answer D1–D7 (defaults apply if you say 'use your recommendations').
- **Step 2:** Claude scaffolds the monorepo, CI and evaluation harness in eagle-eye/, and writes ADR-001 and the interview synthesis template.
- **Step 3:** you begin interviews while Claude builds the crawler and index.
- **Step 4:** at the end of each phase we review the gate evidence together, record the decision, then start the next phase.

> **Principle.** We build one phase at a time and let evidence, not enthusiasm, decide whether to continue. The test for the whole project: can an independent developer put Eagle's Eye on a real website, get trustworthy cited answers, and maintain it with far less effort than building it themselves, and will some of them pay for it?

Prepared 11 October 2026. Timelines, prices, targets and costs are planning hypotheses to be tested. Legal and security items require professional review.